```
"""
بوت تليجرام لتنبيهات أسعار الكريبتو والفوركس
Crypto + Forex Price Alert Telegram Bot
"""

import os
import json
import logging
import requests
from telegram import Update
from telegram.ext import (
    Application,
    CommandHandler,
    ContextTypes,
)

# ============ الإعدادات ============
BOT_TOKEN = os.environ.get("BOT_TOKEN", "")
TWELVEDATA_API_KEY =
os.environ.get("TWELVEDATA_API_KEY", "")
ALERTS_FILE = "alerts.json"
CHECK_INTERVAL_SECONDS = 60  # كل دقيقة

logging.basicConfig(
    format="%(asctime)s - %(name)s - %
(levelname)s - %(message)s",
    level=logging.INFO,
)
logger = logging.getLogger(__name__)
```

```
# قائمة رموز الفوركس الشائعة (لتمييزها عن الكريبتو تلقائياً)
FOREX_SYMBOLS = {
    "EURUSD", "GBPUSD", "USDJPY", "USDCHF",
"AUDUSD",
    "USDCAD", "NZDUSD", "EURGBP", "EURJPY",
"GBPJPY",
}


# ============ إدارة التنبيهات (تخزين في ملف JSON)
============
def load_alerts():
    if not os.path.exists(ALERTS_FILE):
        return []
    with open(ALERTS_FILE, "r") as f:
        return json.load(f)


def save_alerts(alerts):
    with open(ALERTS_FILE, "w") as f:
        json.dump(alerts, f, indent=2)


# ============ جلب الأسعار ============
def get_crypto_price(symbol: str):
    """يجلب سعر عملة كريبتو من Binance (مجاني بدون
مفتاح)"""
    try:
        url =
f"https://api.binance.com/api/v3/ticker/pri
ce?symbol={symbol.upper()}"
        resp = requests.get(url,
timeout=10)
        data = resp.json()
```

```
        if "price" in data:
            return float(data["price"])
        return None
    except Exception as e:
        logger.error(f"Crypto price error
for {symbol}: {e}")
        return None


def get_forex_price(symbol: str):
    """يجلب سعر زوج فوركس من TwelveData (يحتاج
مفتاح مجاني)"""
    try:
        pair = symbol.upper()
        formatted = f"
{pair[:3]}/{pair[3:]}"
        url =
f"https://api.twelvedata.com/price?symbol=
{formatted}&apikey={TWELVEDATA_API_KEY}"
        resp = requests.get(url,
timeout=10)
        data = resp.json()
        if "price" in data:
            return float(data["price"])
        return None
    except Exception as e:
        logger.error(f"Forex price error
for {symbol}: {e}")
        return None


def get_price(symbol: str):
    symbol = symbol.upper()
    if symbol in FOREX_SYMBOLS:
```

```
        return get_forex_price(symbol),
"forex"
    return get_crypto_price(symbol),
"crypto"


# ============ أوامر البوت ============
async def start(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    text = (
        "👋 أهلاً! أنا بوت تنبيهات الأسعار (كريبتو +
فوركس).\n\n"
        "📌 الأوامر المتاحة:\n"
        "/alert SYMBOL PRICE – إضافة تنبيه\n"
        "مثال: /alert BTCUSDT 65000\n"
        "مثال: /alert EURUSD 1.09\n\n"
        "/alerts – عرض تنبيهاتك الحالية\n"
        "/remove ID – حذف تنبيه معين\n"
        "/price SYMBOL – معرفة السعر الحالي\n"
    )
    await update.message.reply_text(text)


async def add_alert(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    chat_id = update.effective_chat.id
    args = context.args

    if len(args) != 2:
        await update.message.reply_text(
            "❌ الصيغة الصحيحة:\n/alert SYMBOL
PRICE\nمثال: /alert BTCUSDT 65000"
        )
        return
```

```
    symbol = args[0].upper()
    try:
        target_price = float(args[1])
    except ValueError:
        await update.message.reply_text("❌ السعر لازم يكون رقم صحيح.")
        return

    current_price, market_type = get_price(symbol)
    if current_price is None:
        await update.message.reply_text(
            f"❌ ما قدرت ألقى سعر لـ {symbol}. تأكد من الرمز صحيح.\n"
            f"كريبتو مثل: BTCUSDT, ETHUSDT\n"
            f"فوركس مثل: EURUSD, GBPUSD"
        )
        return

    direction = "above" if target_price > current_price else "below"

    alerts = load_alerts()
    new_id = (max([a["id"] for a in alerts], default=0)) + 1
    alerts.append({
        "id": new_id,
        "chat_id": chat_id,
        "symbol": symbol,
        "target_price": target_price,
        "direction": direction,
        "market_type": market_type,
    })
```

```
    save_alerts(alerts)

    arrow = "⬆️" if direction == "above"
else "⬇️"
    await update.message.reply_text(
        f"✅ تم إضافة التنبيه #{new_id}\n"
        f"{symbol} ({market_type}) {arrow}
{target_price}\n"
        f"السعر الحالي: {current_price}"
    )


async def list_alerts(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    chat_id = update.effective_chat.id
    alerts = load_alerts()
    user_alerts = [a for a in alerts if
a["chat_id"] == chat_id]

    if not user_alerts:
        await update.message.reply_text("لا
يوجد لديك تنبيهات حالياً.")
        return

    text = "📋 تنبيهاتك الحالية:\n\n"
    for a in user_alerts:
        arrow = "⬆️" if a["direction"] ==
"above" else "⬇️"
        text += f"#{a['id']} —
{a['symbol']} {arrow}
{a['target_price']}\n"
    await update.message.reply_text(text)
```

```
async def remove_alert(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    chat_id = update.effective_chat.id
    args = context.args

    if len(args) != 1:
        await update.message.reply_text("❌
الصيغة الصحيحة:\n/remove ID")
        return

    try:
        alert_id = int(args[0])
    except ValueError:
        await update.message.reply_text("❌
الرقم لازم يكون صحيح.")
        return

    alerts = load_alerts()
    new_alerts = [a for a in alerts if not
(a["id"] == alert_id and a["chat_id"] ==
chat_id)]

    if len(new_alerts) == len(alerts):
        await
update.message.reply_text(f"❌ ما لقيت تنبيه
بالرقم #{alert_id}")
        return

    save_alerts(new_alerts)
    await update.message.reply_text(f"🗑️ تم
حذف التنبيه #{alert_id}")


async def price_command(update: Update,
```

```
context: ContextTypes.DEFAULT_TYPE):
    args = context.args
    if len(args) != 1:
        await update.message.reply_text("❌
الصيغة الصحيحة:\n/price SYMBOL")
        return

    symbol = args[0].upper()
    current_price, market_type =
get_price(symbol)
    if current_price is None:
        await
update.message.reply_text(f"❌ ما قدرت ألقى
سعر لـ {symbol}")
        return

    await update.message.reply_text(f"💰
{symbol} ({market_type}): {current_price}")


# ============ فحص التنبيهات الدوري ============
async def check_alerts(context:
ContextTypes.DEFAULT_TYPE):
    alerts = load_alerts()
    if not alerts:
        return

    remaining_alerts = []
    for alert in alerts:
        current_price, _ =
get_price(alert["symbol"])
        if current_price is None:
            remaining_alerts.append(alert)
            continue
```

```
        triggered = (
            (alert["direction"] == "above"
and current_price >= alert["target_price"])
            or (alert["direction"] ==
"below" and current_price <=
alert["target_price"])
        )

        if triggered:
            try:
                await
context.bot.send_message(

chat_id=alert["chat_id"],
                    text=(
                        f"🚨 تنبيه سعر!\n\n"
                        f"{alert['symbol']}
وصل إلى {current_price}\n"
                        f"(الهدف كان:
{alert['target_price']})"
                    ),
                )
            except Exception as e:
                logger.error(f"Failed to
send alert: {e}")
            # التنبيه يُحذف بعد ما يتفعل مرة وحدة
        else:
            remaining_alerts.append(alert)

    save_alerts(remaining_alerts)


# ============ تشغيل البوت ============
```

```
def main():
    if not BOT_TOKEN:
        raise ValueError("لازم تحط BOT_TOKEN
في environment variables")

    app =
Application.builder().token(BOT_TOKEN).buil
d()

    app.add_handler(CommandHandler("start",
start))
    app.add_handler(CommandHandler("alert",
add_alert))

app.add_handler(CommandHandler("alerts",
list_alerts))

app.add_handler(CommandHandler("remove",
remove_alert))
    app.add_handler(CommandHandler("price",
price_command))

    job_queue = app.job_queue
    job_queue.run_repeating(check_alerts,
interval=CHECK_INTERVAL_SECONDS, first=10)

    logger.info("Bot started...")
    app.run_polling()


if __name__ == "__main__":
    main()
```